BLAZEDEMO AUTOMATION PROJECT CODE

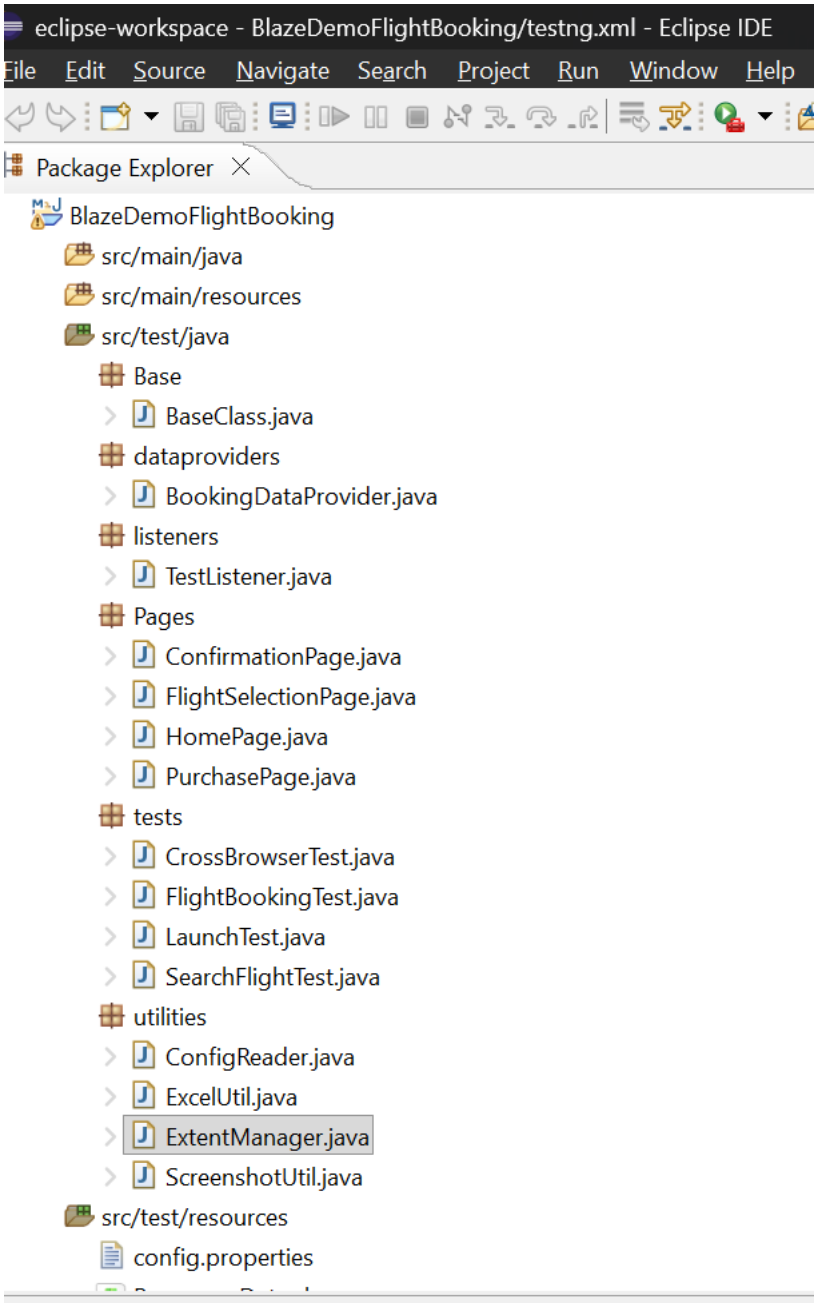
Source Code Documentation

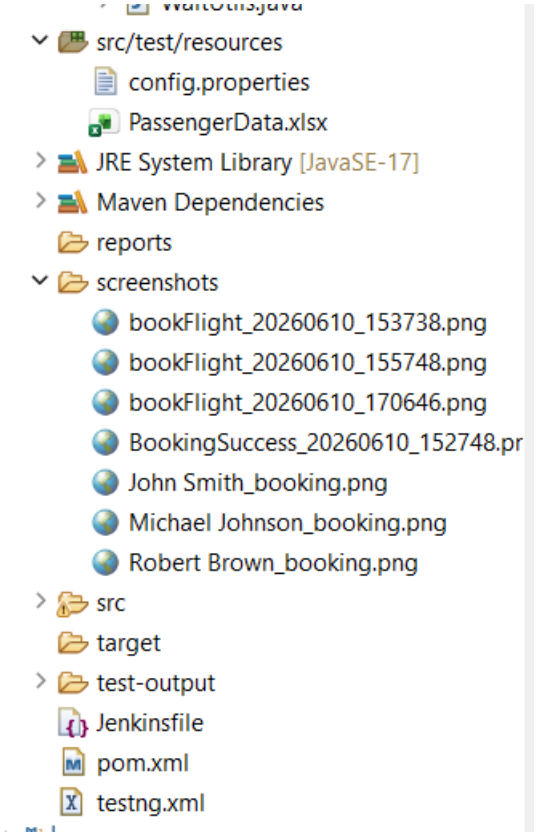
Developed By: Himanshu Sharma

Technology Stack: Java | Selenium WebDriver | TestNG | Maven | Page Object Model (POM) | Jenkins | GitHub | Extent Reports

Project Description: This document contains the complete source code structure, framework architecture, page object classes, utility classes, test scripts, listener implementation, Jenkins pipeline configuration, and reporting mechanism used in the BlazeDemo Flight Booking Automation Project.

1.Structure of Project





Base

BaseClass.java

```
package Base;

import java.time.Duration;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

import org.openqa.selenium.firefox.FirefoxDriver;

import io.github.bonigarcia.wdm.WebDriverManager;

import utilities.ConfigReader;

public class BaseClass {

    public static WebDriver driver;

    // For CrossBrowserTest

    public static void setup(String browser) {

        System.out.println("Launching Browser : " + browser);

        try {

            if (browser.equalsIgnoreCase("chrome")) {
```

```
            WebDriverManager.chromedriver().setup();
            driver = new ChromeDriver();
        } else if (browser.equalsIgnoreCase("firefox")) {

            WebDriverManager.firefoxdriver().setup();
            driver = new FirefoxDriver();
        } else {
            System.out.println(
                    "Invalid browser. Launching Chrome by default.");
            WebDriverManager.chromedriver().setup();
            driver = new ChromeDriver();
        }
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(
                Duration.ofSeconds(
                        Long.parseLong(
                                ConfigReader.getProperty("implicitWait"))));
        driver.get(ConfigReader.getProperty("url"));
    } catch (Exception e) {
        System.out.println("Browser Launch Failed");
        e.printStackTrace();
    }
}
// For normal tests (LaunchTest, SearchFlightTest, FlightBookingTest)
public static void setup() {
    setup(ConfigReader.getProperty("browser"));
}
public static void tearDown() {
    if (driver != null) {
        driver.quit();
```

```
            driver = null;
        }
    }
}
```

dataproviders

BookingDataProvider.java

```
package dataproviders;
import org.testng.annotations.DataProvider;
import utilities.ExcelUtil;
public class BookingDataProvider {
    @DataProvider(name = "bookingData")
    public Object[][] getBookingData() {
        return ExcelUtil.getExcelData();
    }
}
```

listeners

TestListener.java

```
package listeners;
import org.testng.ITestContext;
import org.testng.ITestListener;
import org.testng.ITestResult;
import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;
import utilities.ExtentManager;
import utilities.ScreenshotUtil;
public class TestListener implements ITestListener {
    ExtentReports extent = ExtentManager.getReport();
    ExtentTest test;
    @Override
```

```
    public void onTestStart(ITestResult result) {
        test = extent.createTest(result.getName());

    @Override
    public void onTestSuccess(ITestResult result) {
        ScreenshotUtil.captureScreenshot(result.getName() + "_PASSED");
        if (test != null) {
            test.pass("Test Passed");
        }
    }
    @Override
    public void onTestFailure(ITestResult result) {
        ScreenshotUtil.captureScreenshot(result.getName() + "_FAILED");
        if (test != null) {
            test.fail(result.getThrowable());
        }
    }
    @Override
    public void onTestSkipped(ITestResult result) {
        if (test != null) {
            test.skip("Test Skipped");
        }
    }
    @Override
    public void onFinish(ITestContext context) {
        extent.flush();
    }
}
```

Pages

ConfirmationPage.java

```
package Pages;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
public class ConfirmationPage {
   WebDriver driver;
   public ConfirmationPage(WebDriver driver) {
      this.driver = driver;
   }
   By confirmationMsg =
         By.tagName("h1");
   public String getConfirmationMessage() {
      return driver.findElement(confirmationMsg)
               .getText();
   }
}
```

FlightSelectionPage.java

```
package Pages;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
public class FlightSelectionPage {
   WebDriver driver;
   public FlightSelectionPage(WebDriver driver) {
      this.driver = driver;
   }
   By flightsTable =
         By.xpath("//table");
   public boolean flightsDisplayed() {
      return driver.findElement(flightsTable)
```

```
            .isDisplayed();
    }
    public void chooseFlight(int flightIndex)
            throws InterruptedException {
        driver.findElement(
                By.xpath("(//input[@value='Choose This Flight'])["
                + flightIndex + "]"))
                .click();
        Thread.sleep(2000);
    }
}
```

HomePage.java

```
package Pages;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.Select;
public class HomePage {
    WebDriver driver;
    public HomePage(WebDriver driver) {
        this.driver = driver;
    }
    By departureCity =
            By.name("fromPort");
    By destinationCity =
            By.name("toPort");
    By findFlightsBtn =
            By.cssSelector("input[type='submit']");
    public void selectDeparture(String city) {
        Select dep =
```

```
            new Select(driver.findElement(departureCity));
        dep.selectByVisibleText(city);
    }
 public void selectDestination(String city) {
        Select dest =
            new Select(driver.findElement(destinationCity));
        dest.selectByVisibleText(city);
    }
    public void clickFindFlights() {
        driver.findElement(findFlightsBtn).click();
    }
}
```

PurchasePage.java

```
package Pages;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.Select;
public class PurchasePage {
    WebDriver driver;
    public PurchasePage(WebDriver driver) {
        this.driver = driver;
    }
    By totalCost =
        By.xpath("//p[contains(text(),'Price')]");
    By name =
        By.id("inputName");
    By address =
        By.id("address");
    By city =
```

```
            By.id("city");
    By state =
            By.id("state");
By zipCode =
           By.id("zipCode");
    By cardType =
            By.id("cardType");
    By cardNumber =
            By.id("creditCardNumber");
    By month =
            By.id("creditCardMonth");
    By year =
            By.id("creditCardYear");
    By nameOnCard =
            By.id("nameOnCard");
    By purchaseBtn =
            By.cssSelector("input[type='submit']");
    public boolean verifyCostDisplayed() {
       return driver.findElement(totalCost)
               .isDisplayed();
    }
    public void enterPassengerDetails(
            String passengerName,
            String passengerAddress,
            String passengerCity,
            String passengerState,
            String passengerZip,
            String cardTypeValue,
            String cardNo,
            String monthValue,
```

```
    String yearValue,
    String cardHolderName)
    throws InterruptedException {

driver.findElement(name)
        .sendKeys(passengerName);
Thread.sleep(2000);
driver.findElement(address)
        .sendKeys(passengerAddress);
Thread.sleep(2000);
driver.findElement(city)
        .sendKeys(passengerCity);
Thread.sleep(2000);
driver.findElement(state)
        .sendKeys(passengerState);
Thread.sleep(2000);
driver.findElement(zipCode)
        .sendKeys(passengerZip);
Thread.sleep(2000);
Select select =
        new Select(driver.findElement(cardType));
select.selectByVisibleText(cardTypeValue);
Thread.sleep(2000);
driver.findElement(cardNumber)
        .sendKeys(cardNo);
Thread.sleep(2000);
driver.findElement(month)
        .clear();
Thread.sleep(1000);
driver.findElement(month)
```

```
            .sendKeys(monthValue);
        Thread.sleep(2000);
        driver.findElement(year)
            .clear();
        Thread.sleep(1000);
        driver.findElement(year)
            .sendKeys(yearValue);
        Thread.sleep(2000);
        driver.findElement(nameOnCard)
            .sendKeys(cardHolderName);
        Thread.sleep(2000);
    }
    public void clickPurchaseFlight() {
        driver.findElement(purchaseBtn)
            .click();
    }
}
```

tests

CrossBrowserTest.java

```
package tests;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Listeners;
import org.testng.annotations.Parameters;
import org.testng.annotations.Test;
import Base.BaseClass;
@Listeners(listeners.TestListener.class)
public class CrossBrowserTest extends BaseClass {
```

```
    @Parameters("browser")
    @BeforeMethod
    public void launchBrowser(String browser) {
   setup(browser);
    }
    @Test
    public void verifyApplicationOnDifferentBrowsers()
            throws InterruptedException {
        Thread.sleep(3000);
        Assert.assertEquals(driver.getTitle(), "BlazeDemo");
        System.out.println("Application launched successfully");
        Thread.sleep(2000);
    }
    @AfterMethod
    public void closeBrowser() {
        tearDown();
    }
}
```

FlightBookingTest.java

```
package tests;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Listeners;
import org.testng.annotations.Test;
import Base.BaseClass;
import Pages.ConfirmationPage;
import Pages.FlightSelectionPage;
import Pages.HomePage;
```

```
import Pages.PurchasePage;
import dataproviders.BookingDataProvider;
@Listeners(listeners.TestListener.class)
public class FlightBookingTest extends BaseClass {
    HomePage home;
    FlightSelectionPage flight;
    PurchasePage purchase;
    ConfirmationPage confirmation;
    @BeforeMethod
    public void launchBrowser() {
        setup();
    }
    @Test(
            dataProvider = "bookingData",
            dataProviderClass = BookingDataProvider.class
    )
    public void verifyFlightBooking(
            String departure,
            String destination,
            String passengerName,
            String address,
            String city,
            String state,
            String zip,
            String cardType,
            String cardNumber,
            String month,
            String year,
            String nameOnCard,
            String flightIndex)
```

```
    throws InterruptedException {
// Home Page
home = new HomePage(driver);

home.selectDeparture(departure);
Thread.sleep(2000);
home.selectDestination(destination);
Thread.sleep(2000);
home.clickFindFlights();
Thread.sleep(3000);
// Flight Selection Page
flight = new FlightSelectionPage(driver);
Assert.assertTrue(
        flight.flightsDisplayed(),
        "Flights are not displayed");
Thread.sleep(2000);
flight.chooseFlight(Integer.parseInt(flightIndex));
Thread.sleep(3000);
// Purchase Page
purchase = new PurchasePage(driver);
Assert.assertTrue(
        purchase.verifyCostDisplayed(),
        "Purchase page is not displayed");
Thread.sleep(2000);
purchase.enterPassengerDetails(
        passengerName,
        address,
        city,
        state,
        zip,
```

```
                cardType,
                cardNumber,
                month,
                year,
                nameOnCard);
        Thread.sleep(3000);
        purchase.clickPurchaseFlight();
        Thread.sleep(5000);
        // Confirmation Page
        confirmation = new ConfirmationPage(driver);
        String actualMessage =
                confirmation.getConfirmationMessage();
        Assert.assertEquals(
                actualMessage,
                "Thank you for your purchase today!");
        Thread.sleep(2000);
        System.out.println(
                "Flight Booking Completed Successfully");
        Thread.sleep(2000);
    }
    @AfterMethod
    public void closeBrowser() {
        tearDown();
    }
}
```

LaunchTest.java

```
package tests;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
```

```
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Listeners;
import org.testng.annotations.Test;
import Base.BaseClass;
@Listeners(listeners.TestListener.class)
public class LaunchTest extends BaseClass {
   @BeforeMethod
   public void launchBrowser() {
      setup();
   }
   @Test
   public void verifyApplicationLaunch() throws InterruptedException {
      Thread.sleep(2000);
      String actualTitle = driver.getTitle();
      Assert.assertEquals(
            actualTitle,
            "BlazeDemo");
      System.out.println(
            "Application launched successfully");
      Thread.sleep(2000);
   }
   @AfterMethod
   public void closeBrowser() {
      tearDown();
   }
}
```

SearchFlightTest.java

```
package tests;
import org.testng.Assert;
```

```
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Listeners;
import org.testng.annotations.Test;
import Base.BaseClass;
import Pages.FlightSelectionPage;
import Pages.HomePage;
import dataproviders.BookingDataProvider;
@Listeners(listeners.TestListener.class)
public class SearchFlightTest extends BaseClass {
    HomePage home;
    FlightSelectionPage flight;
    @BeforeMethod
    public void launchBrowser() {
        setup();
    }
    @Test(
            dataProvider = "bookingData",
            dataProviderClass = BookingDataProvider.class
    )
    public void verifyFlightSearch(
            String departure,
            String destination,
            String passengerName,
            String address,
            String city,
            String state,
            String zip,
            String cardType,
            String cardNumber,
```

```
            String month,
            String year,
            String nameOnCard,
            String flightIndex)
            throws InterruptedException {
        home = new HomePage(driver);
        home.selectDeparture(departure);
        Thread.sleep(2000);
        home.selectDestination(destination);
        Thread.sleep(2000);
        home.clickFindFlights();
        Thread.sleep(3000);
        flight = new FlightSelectionPage(driver);
        Assert.assertTrue(
                flight.flightsDisplayed(),
                "Flights are not displayed");
        System.out.println(
                "Flight search completed successfully");
        Thread.sleep(2000);
    }
    @AfterMethod
    public void closeBrowser() {
        tearDown();
    }
}
```

utilities

ConfigReader.java

```
package utilities;
import java.io.FileInputStream;
```

```
import java.io.IOException;
import java.util.Properties;
public class ConfigReader {
private static Properties prop = new Properties();
   static {
      try {
         FileInputStream fis =
               new FileInputStream("src/test/resources/config.properties");
         prop.load(fis);
         fis.close();
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
   public static String getProperty(String key) {
      return prop.getProperty(key);
   }
}
```

ExcelUtil.java

```
package utilities;
import java.io.FileInputStream;
import java.io.IOException;
import org.apache.poi.ss.usermodel.DataFormatter;
import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
public class ExcelUtil {
   public static Object[][] getExcelData() {
      Object[][] data = null;
      try {
```

```
    String excelPath =
            ConfigReader.getProperty("excelPath");
    FileInputStream fis =
            new FileInputStream(excelPath);
    XSSFWorkbook workbook =
            new XSSFWorkbook(fis);
    XSSFSheet sheet =
            workbook.getSheetAt(0);
    DataFormatter formatter =
            new DataFormatter();
    int rowCount =
            sheet.getLastRowNum();
    int colCount =
            sheet.getRow(0).getLastCellNum();
    data = new Object[rowCount][colCount];
    for (int i = 1; i <= rowCount; i++) {
       for (int j = 0; j < colCount; j++) {
          if (sheet.getRow(i) != null
                && sheet.getRow(i).getCell(j) != null) {
             data[i - 1][j] =
                   formatter.formatCellValue(
                         sheet.getRow(i).getCell(j));
          } else {
             data[i - 1][j] = "";
          }
       }
    }
    workbook.close();
    fis.close();
} catch (IOException e) {
```

```
            e.printStackTrace();
        }
        return data;
    }}
```

ExtentManager.java

```
package utilities;
import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;
public class ExtentManager {
    private static ExtentReports extent;
    public static ExtentReports getReport() {
        if (extent == null) {
            ExtentSparkReporter spark =
                    new ExtentSparkReporter("Reports/ExtentReport.html");
            spark.config().setReportName("BlazeDemo Flight Booking Report");
            spark.config().setDocumentTitle("Automation Report");
            extent = new ExtentReports();
            extent.attachReporter(spark);
                extent.setSystemInfo("Tester", "Himanshu Sharma");
                extent.setSystemInfo("Framework", "Selenium + TestNG");
                extent.setSystemInfo("Project", "BlazeDemo Automation");

        }
        return extent;
    }
}
```

ScreenshotUtil.java

```
package utilities;
import java.io.File;
```

```
import java.io.IOException;
import java.text.SimpleDateFormat;
import java.util.Date;
import org.apache.commons.io.FileUtils;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import Base.BaseClass;
public class ScreenshotUtil extends BaseClass {
   public static void captureScreenshot(String testName) {
      try {
         if (driver == null) {
            System.out.println(
                  "Driver is null. Screenshot skipped.");
            return;
         }
         TakesScreenshot ts =
               (TakesScreenshot) driver;
         File source =
               ts.getScreenshotAs(OutputType.FILE);
         String timeStamp =
               new SimpleDateFormat("yyyyMMdd_HHmmss")
                     .format(new Date());
         File destination =
               new File("./Screenshots/"
                     + testName + "_"
                     + timeStamp + ".png");
         FileUtils.copyFile(source, destination);
         System.out.println(
               "Screenshot saved successfully.");
      } catch (IOException e) {
```

```
            e.printStackTrace();
        } catch (Exception e) {
            System.out.println(
                    "Screenshot capture failed: "
                            + e.getMessage());}}}
```

config.properties

```
browser=chrome
url=https://blazedemo.com/
excelPath=src/test/resources/PassengerData.xlsx
implicitWait=10
```

JenkinsFile

```
pipeline {
    agent any
    tools {
        jdk 'JDK'
        maven 'Maven'
    }
    stages {
        stage('Checkout Code') {
            steps {
                git branch: 'main',
                url: 'https://github.com/Himanshu0320/Capstone-Project '
            }
        }
        stage('Clean Project') {
            steps {
                bat 'mvn clean'
            }
        }
```

```
        stage('Compile Project') {
            steps {
                bat 'mvn compile'
            }
        }
        stage('Execute TestNG Tests') {
            steps {
                bat 'mvn test -DexecutionMode=jenkins'
            }
        }
        stage('Archive Reports') {
            steps {
                archiveArtifacts artifacts: 'test-output/**/*.*', allowEmptyArchive: true
                archiveArtifacts artifacts: 'Screenshots/**/*.*', allowEmptyArchive: true
                archiveArtifacts artifacts: 'Reports/**/*.*', allowEmptyArchive: true
            }
        }
    }
    post {
        success {
            echo 'BlazeDemo Automation Build Successful'
        }
        failure {
            echo 'BlazeDemo Automation Build Failed'
        }
        always {
            echo 'Pipeline Execution Completed'
        }
    }
}
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.capstone</groupId>
    <artifactId>BlazeDemoFlightBooking</artifactId>
    <version>1.0</version>
    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.22.0</version>
        </dependency>
        <dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>5.9.2</version>
</dependency>
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.10.2</version>
            <scope>test</scope>
        </dependency>
        <dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.16.1</version>
```

```
</dependency>
<dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi</artifactId>
    <version>5.2.5</version>
</dependency>
<dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>5.2.5</version>
</dependency>
<dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>5.1.2</version>
</dependency>
    </dependencies>
    <build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.2.5</version>
            <configuration>
                <suiteXmlFiles>
                    <suiteXmlFile>testng.xml</suiteXmlFile>
                </suiteXmlFiles>
            </configuration>
        </plugin>
    </plugins>
```

```
</build>
</project>
```

testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="BlazeDemoSuite">
    <listeners>
        <listener class-name="Listeners.TestListener"/>
    </listeners>
    <test name="Chrome Test">
        <parameter name="browser" value="chrome"/>
        <classes>
            <class name="tests.FlightBookingTest"/>
        </classes>
    </test>
    <test name="Firefox Test">
        <parameter name="browser" value="firefox"/>
        <classes>
            <class name="tests.FlightBookingTest"/>
        </classes>
    </test>
</suite>
```